
High-Level Architecture

Your script is an interactive Computer Vision toolkit built primarily on **MediaPipe's Tasks API** (with a little help from deepface). It is divided into four main modules, all orchestrated by a central `main_menu()`.

Each module generally follows the same pipeline:

1. **Initialize Model:** Load the .task file (e.g., `face_landmarker.task`).
 2. **Capture Input:** Read from a webcam, video, or static image.
 3. **Process Frame:** Convert the image to RGB, pass it to MediaPipe, and extract specific coordinates (landmarks).
 4. **Apply Logic:** Calculate distances or states based on those coordinates.
 5. **Render UI:** Draw bounding boxes, text, and lines on the frame using OpenCV (`cv2`).
-

Module Breakdown & The Math Behind Them

1. Lips Detection (Smile & Mouth Aspect Ratio)

- **Core Concept:** This module determines if the mouth is open (yawning/talking) and if the person is smiling.
- **The Math (MAR):** It calculates the Mouth Aspect Ratio (MAR).
 - $MAR = \frac{\text{Vertical Distance}}{\text{Horizontal Distance}}$
 - It measures the distance between the upper and lower lips, divided by the distance between the left and right corners. If $MAR > 0.55$, the script assumes the mouth is open.
- **Smile Logic:** It checks the y -coordinates. In images, the y -axis increases as you go *down*. If the corners of the mouth are higher (smaller y value) than the center of the lips, the person is smiling.

2. Eyes Detection (Blink & Drowsiness)

- **Core Concept:** Tracks blinks and triggers an alert if eyes remain closed too long.
- **The Math (EAR):** It calculates the Eye Aspect Ratio (EAR) using 6 landmarks around each eye.
 - $EAR = \frac{A + B}{2C}$ where A and B are the vertical distances between the eyelids, and C is the horizontal distance from corner to corner.
 - If $EAR < 0.25$, the eye is considered closed. If it stays closed for 20 consecutive frames, the "DROWSY" alert triggers.

3. Face Detection (Emotion & Pose)

- **Core Concept:** Estimates where the head is looking and guesses the user's emotional state.

- **Head Pose Logic:** It compares the x and y coordinates of the nose tip to the midpoint between the eyes. If the nose is significantly far to the left/right/up/down of the eye midpoint, it registers a turn.
- **Emotion Logic (Hybrid Engine):** This is the most complex part. It uses deepface for heavy emotional analysis every 10 frames (because DeepFace is slow). In between, it uses MediaPipe's **Blendshapes** (facial muscle movements like BS_JAW_OPEN or BS_BROW_INNER_UP) to boost or confirm the emotions instantly.

4. Hand Detection (Gestures & Finger Counting)

- **Core Concept:** Counts how many fingers are held up and runs a mini-game.
- **Finger Counting Logic:** It compares the distance of the finger's tip to the wrist ($\text{Landmark} \setminus 0$) versus the finger's middle joint (PIP) to the wrist. If the tip is further from the wrist than the middle joint, the finger is extended.

Heavily Commented Code Reference

Save this version to use during your evaluation. I have added block comments explaining the logic and inline comments explaining the specific variables and indices.

Python

```
import cv2
import numpy as np
import math
import os
import mediapipe as mp
# Importing specific submodules from MediaPipe's newer Tasks API
from mediapipe.tasks import python
from mediapipe.tasks.python import vision
from mediapipe.tasks.python.core import base_options

# =====
# MODULE 1: LIPS DETECTION (Mouth Aspect Ratio & Smile)
# =====
def lips_detection_module():
    """
    Module 1: Lips Detection
    Contains all logic for Smile Detection, MAR Tracking, and Lip-Sync Counting.
    Includes internal menu for selecting input source.
    """
```

```

# Path to the pre-trained MediaPipe face landmarking model
MODEL_ASSET_PATH = "face_landmarker.task"

# Setup shortcuts for MediaPipe API to make code cleaner
BaseOptions = base_options.BaseOptions
FaceLandmarker = vision.FaceLandmarker
FaceLandmarkerOptions = vision.FaceLandmarkerOptions
VisionRunningMode = vision.RunningMode

def calculate_distance(p1, p2):
    """Calculates the Euclidean distance between two 2D points (x, y)."""
    # math.hypot calculates the hypotenuse of a right triangle, which is standard distance.
    return math.hypot(p2[0] - p1[0], p2[1] - p1[1])

def get_mouth_metrics(face_landmarks_list, w, h):
    """
    Extracts specific mouth landmarks and calculates MAR and Smile status.
    w, h are the width and height of the image frame to convert normalized coords to pixels.
    """
    def get_coords(index):
        # MediaPipe returns values from 0.0 to 1.0. Multiply by width/height to get pixels.
        pt = face_landmarks_list[index]
        return (int(pt.x * w), int(pt.y * h))

    # 13 and 14 are the inner center of the upper and lower lips
    p_upper, p_lower = get_coords(13), get_coords(14)
    # 78 and 308 are the left and right corners of the mouth
    p_left, p_right = get_coords(78), get_coords(308)

    # Calculate vertical (lip to lip) and horizontal (corner to corner) distances
    vert_dist = calculate_distance(p_upper, p_lower)
    horiz_dist = calculate_distance(p_left, p_right)

    # MAR = Vertical / Horizontal. Protect against division by zero.
    mar = vert_dist / horiz_dist if horiz_dist > 0 else 0.0

    # Smile detection logic: Y-axis goes DOWN in OpenCV.
    # If the corners of the mouth are physically HIGHER (lower Y value) than the center of the lips, it's a
    smile.
    mouth_center_y = (p_upper[1] + p_lower[1]) / 2
    corners_y = (p_left[1] + p_right[1]) / 2
    is_smiling = (mouth_center_y - corners_y) > 2.0

```

```
return mar, is_smiling, [p_upper, p_lower, p_left, p_right]
```

```
def module1_lips_detection_image(image_path):
```

```
    # Implementation for static images (simplified version of the video loop below)
```

```
    if not os.path.exists(image_path):
```

```
        print(f"Error: Path '{image_path}' not found.")
```

```
    return
```

```
frame = cv2.imread(image_path)
```

```
if frame is None:
```

```
    print("Error: Could not read image.")
```

```
return
```

```
h, w, _ = frame.shape
```

```
# Configure MediaPipe for IMAGE mode (static, no tracking over time)
```

```
options = FaceLandmarkerOptions(
```

```
    base_options=BaseOptions(model_asset_path=MODEL_ASSET_PATH),
```

```
    running_mode=VisionRunningMode.IMAGE
```

```
)
```

```
with FaceLandmarker.create_from_options(options) as landmarker:
```

```
    # MediaPipe requires RGB format, OpenCV defaults to BGR
```

```
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB,  
data=cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
```

```
    results = landmarker.detect(mp_image)
```

```
smile_label = "Neutral"
```

```
mar_display = 0.0
```

```
draw_red_border = False
```

```
# If a face is found, process the first one (index [0])
```

```
if results.face_landmarks:
```

```
    mar, is_smiling, pts = get_mouth_metrics(results.face_landmarks[0], w, h)
```

```
    mar_display = mar
```

```
    # Threshold for mouth open is 0.55
```

```
    if mar > 0.55: draw_red_border = True
```

```
    if is_smiling: smile_label = "Smiling"
```

```
    # Draw small green circles on the 4 mouth landmarks
```

```
    for pt in pts: cv2.circle(frame, pt, 2, (0, 255, 0), -1)
```

```
if draw_red_border:
```

```
    cv2.rectangle(frame, (0, 0), (w - 1, h - 1), (0, 0, 255), 10)
```

```

    # Display text on screen
    cv2.putText(frame, f"Expression: {smile_label}", (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.7,
(255, 255, 0), 2)
    cv2.putText(frame, f"MAR: {mar_display:.2f}", (20, 80), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,
255, 255), 2)
    cv2.imshow("Module 1: Lips Detection - Image", frame)
    cv2.waitKey(0) # Wait infinitely until user presses a key
    cv2.destroyAllWindows()

```

```

def module1_lips_detection(input_source=0):
    # Implementation for Live Video/Webcam
    cap = cv2.VideoCapture(input_source)
    if not cap.isOpened():
        print("Error: Could not open video source.")
    return

```

```

    # Get Frame Per Second (FPS) to help calculate accurate timestamps for MediaPipe
    fps = cap.get(cv2.CAP_PROP_FPS)
    if fps <= 0: fps = 30.0
    frame_counter = 0

```

```

    # Configure for VIDEO mode (tracks faces across frames for better performance)
    options = FaceLandmarkerOptions(
        base_options=BaseOptions(model_asset_path=MODEL_ASSET_PATH),
        running_mode=VisionRunningMode.VIDEO
    )

```

```

    is_mouth_open = False
    cycle_count = 0 # Tracks how many times mouth opened and closed
    paused = False

```

```

    with FaceLandmarker.create_from_options(options) as landmarker:
        while cap.isOpened():
            if not paused:
                success, frame = cap.read()
                if not success:
                    # If video ends, loop it back to frame 0
                    if isinstance(input_source, str):
                        cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
                    continue
                break

```

```

h, w, _ = frame.shape

# Generate a strictly increasing timestamp required by Video mode
timestamp_ms = int((1000 / fps) * frame_counter)
frame_counter += 1

mp_image = mp.Image(image_format=mp.ImageFormat.SRGB,
data=cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
results = landmarker.detect_for_video(mp_image, timestamp_ms)

draw_red_border = False
mar_display = 0.0
smile_label = "Neutral"

if results.face_landmarks:
    mar, is_smiling, _ = get_mouth_metrics(results.face_landmarks[0], w, h)
    mar_display = mar

# Logic for counting open/close cycles (hysteresis to prevent flickering)
if mar > 0.55:
    draw_red_border = True
    is_mouth_open = True
# Only count a cycle if MAR drops below 0.35 AFTER being open
elif mar < 0.35 and is_mouth_open:
    cycle_count += 1
    is_mouth_open = False

if is_smiling: smile_label = "Smiling"

# UI overlays
if draw_red_border:
    cv2.rectangle(frame, (0, 0), (w - 1, h - 1), (0, 0, 255), 10)

    cv2.putText(frame, f"Expression: {smile_label}", (20, 40), cv2.FONT_HERSHEY_SIMPLEX,
0.7, (255, 255, 0), 2)
    cv2.putText(frame, f"MAR: {mar_display:.2f}", (20, 80), cv2.FONT_HERSHEY_SIMPLEX,
0.7, (0, 255, 255), 2)
    cv2.putText(frame, f"Cycles: {cycle_count}", (20, 120), cv2.FONT_HERSHEY_SIMPLEX,
0.7, (0, 255, 0), 2)
    cv2.imshow("Module 1: Lips Detection", frame)

# Keyboard controls
key = cv2.waitKey(1) & 0xFF

```

```

        if key == 27: # ESC key to exit
            break
        elif key == ord("p"): # Pause
            paused = not paused
        elif key == ord("r"): # Reset video and counter
            cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
            cycle_count = 0

    cap.release()
    cv2.destroyAllWindows()

# --- Internal Menu for Module 1 ---
if not os.path.exists(MODEL_ASSET_PATH):
    print(f"Error: Face Landmarker model '{MODEL_ASSET_PATH}' not found.")
    return

while True:
    print("\n--- Lips Detection Module Menu ---")
    print("1. Live Webcam Feed\n2. Video File Upload\n3. Image Upload\n0. Back to Main Menu")
    choice = input("Select Source: ")
    if choice == "0": break
    elif choice == "1": module1_lips_detection(0)
    elif choice == "2": module1_lips_detection(input("Enter Video Path: ").strip())
    elif choice == "3": module1_lips_detection_image(input("Enter Image Path: ").strip())
    else: print("Invalid Choice.")

# =====
# MODULE 2: EYES DETECTION (Eye Aspect Ratio, Blinks, Drowsiness)
# =====
def eyes_detection_module():
    import cv2
    import numpy as np
    import mediapipe as mp
    import time
    import os
    from mediapipe.tasks import python
    from mediapipe.tasks.python import vision
    from mediapipe.tasks.python.core import base_options

    # --- Global Constants for Eye Logic ---
    EAR_THRESHOLD_CLOSED = 0.25 # If EAR is below 0.25, eye is considered closed
    FRAMES_PER_SECOND = 30

```

```
DROWSY_FRAMES_THRESHOLD = 20 # Must be closed for 20 continuous frames to trigger alert
CLEAR_DROWSY_FRAMES_THRESHOLD = 5 # Must be open for 5 continuous frames to clear the
alert
```

```
MODEL_ASSET_PATH = "face_landmarker.task"
if not os.path.exists(MODEL_ASSET_PATH): return
```

```
FaceLandmarkerOptions = vision.FaceLandmarkerOptions
BaseOptions = base_options.BaseOptions
```

```
# We don't need blendshapes here, so turn them off to save processing power
options = FaceLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=MODEL_ASSET_PATH),
    output_face_blendshapes=False,
    output_facial_transformation_matrixes=False,
    num_faces=1
)
```

```
# Specific point indices from MediaPipe's dense 468-point face mesh for the eyes
LEFT_EYE_LANDMARKS = [362, 385, 387, 263, 373, 380]
RIGHT_EYE_LANDMARKS = [33, 160, 158, 133, 153, 144]
```

```
# --- Helper Functions ---
```

```
def euclidean_distance(point1, point2):
    # Uses numpy to calculate distance between two arrays/points
    return np.linalg.norm(np.array(point1) - np.array(point2))
```

```
def calculate_ear(eye_landmarks, face_landmarks_list, img_w, img_h):
    """Calculates the Eye Aspect Ratio using 6 points around the eye contour."""
    if not all(idx < len(face_landmarks_list) for idx in eye_landmarks):
        return 0.0
    p = []
    for i in eye_landmarks:
        landmark = face_landmarks_list[i]
        x, y = int(landmark.x * img_w), int(landmark.y * img_h)
        p.append((x, y))
```

```
# P[1] to P[5] and P[2] to P[4] are the vertical distances
A = euclidean_distance(p[1], p[5])
B = euclidean_distance(p[2], p[4])
# P[0] to P[3] is the horizontal distance across the eye
C = euclidean_distance(p[0], p[3])
```



```
# EAR Formula. Divides by 2.0*C to normalize it.
```

```
return (A + B) / (2.0 * C) if C != 0 else 0.0
```

```
def process_frame_eyes_detection(frame, face_landmarker_detector, blink_counter_state, drowsy_state):
```

```
    img_h, img_w, _ = frame.shape
```

```
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=frame_rgb)
```

```
    results = face_landmarker_detector.detect(mp_image)
```

```
    left_ear, right_ear = 0.0, 0.0
```

```
    if results.face_landmarks:
```

```
        landmarks_list = results.face_landmarks[0]
```

```
        left_ear = calculate_ear(LEFT_EYE_LANDMARKS, landmarks_list, img_w, img_h)
```

```
        right_ear = calculate_ear(RIGHT_EYE_LANDMARKS, landmarks_list, img_w, img_h)
```

```
    # BLINK LOGIC
```

```
    # If both eyes are below threshold, mark as closed
```

```
    if left_ear < EAR_THRESHOLD_CLOSED and right_ear < EAR_THRESHOLD_CLOSED:
```

```
        blink_counter_state['is_eye_closed'] = True
```

```
    # If they WERE closed, and are now open, count it as exactly 1 blink
```

```
    elif blink_counter_state['is_eye_closed'] and left_ear > EAR_THRESHOLD_CLOSED and right_ear > EAR_THRESHOLD_CLOSED:
```

```
        blink_counter_state['blink_count'] += 1
```

```
        blink_counter_state['is_eye_closed'] = False
```

```
    # DROWSINESS LOGIC
```

```
    if left_ear < EAR_THRESHOLD_CLOSED and right_ear < EAR_THRESHOLD_CLOSED:
```

```
        drowsy_state['consecutive_closed_frames'] += 1
```

```
        drowsy_state['consecutive_open_frames'] = 0 # Reset open counter
```

```
    else:
```

```
        drowsy_state['consecutive_open_frames'] += 1
```

```
    # Only turn off the alarm if eyes have been open for a while
```

```
    if drowsy_state['consecutive_open_frames'] >= CLEAR_DROWSY_FRAMES_THRESHOLD:
```

```
        drowsy_state['is_drowsy_alert_active'] = False
```

```
        drowsy_state['consecutive_closed_frames'] = 0
```

```
    # Trigger alarm if closed for too long
```

```
    if drowsy_state['consecutive_closed_frames'] >= DROWSY_FRAMES_THRESHOLD:
```

```
        drowsy_state['is_drowsy_alert_active'] = True
```

```
    # Draw dots on the eyes
```

```

        for eye_indices in [LEFT_EYE_LANDMARKS, RIGHT_EYE_LANDMARKS]:
            for idx in eye_indices:
                landmark = landmarks_list[idx]
                cv2.circle(frame, (int(landmark.x * img_w), int(landmark.y * img_h)), 2, (0, 255, 255), -1)

            cv2.putText(frame, f"L-EAR: {left_ear:.3f}", (10, img_h - 60), cv2.FONT_HERSHEY_SIMPLEX,
0.7, (0, 255, 0), 2)
            cv2.putText(frame, f"R-EAR: {right_ear:.3f}", (10, img_h - 30), cv2.FONT_HERSHEY_SIMPLEX,
0.7, (0, 255, 0), 2)

        cv2.putText(frame, f"Blinks: {blink_counter_state['blink_count']}", (img_w - 180, 30),
cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 0), 2)

        # Drowsiness visual alert (red screen overlay)
        if drowsy_state['is_drowsy_alert_active']:
            overlay = frame.copy()
            cv2.rectangle(overlay, (0, img_h // 2 - 50), (img_w, img_h // 2 + 50), (0, 0, 255), -1)
            # Alpha blending for transparency
            frame = cv2.addWeighted(overlay, 0.6, frame, 0.4, 0)
            cv2.putText(frame, 'DROWSY! Wake Up!', (img_w // 2 - 200, img_h // 2 + 15),
cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 3)

    return frame, left_ear, right_ear

# --- Runners (Webcam, Video, Image) ---
def run_webcam(detector):
    cap = cv2.VideoCapture(0)
    # Using dictionaries to pass states by reference so they persist across frames
    blink_state = {'blink_count': 0, 'is_eye_closed': False}
    drowsy_state = {'consecutive_closed_frames': 0, 'consecutive_open_frames': 0,
'is_drowsy_alert_active': False}
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret: break
        frame = cv2.flip(frame, 1) # Mirror effect
        annotated, _, _ = process_frame_eyes_detection(frame, detector, blink_state,
drowsy_state)
        cv2.imshow('Eyes Detection', annotated)
        if cv2.waitKey(1) & 0xFF == 27: break
    cap.release()
    cv2.destroyAllWindows()

def run_video(path, detector):

```

```

cap = cv2.VideoCapture(path)
paused, current_pos = False, 0
blink_state = {'blink_count': 0, 'is_eye_closed': False}
drowsy_state = {'consecutive_closed_frames': 0, 'consecutive_open_frames': 0,
'is_drowsy_alert_active': False}
while True:
    if not paused:
        cap.set(cv2.CAP_PROP_POS_FRAMES, current_pos)
        ret, frame = cap.read()
        if not ret: paused = True; continue
        current_pos += 1
        annotated, _, _ = process_frame_eyes_detection(frame, detector, blink_state,
drowsy_state)
        cv2.imshow('Eyes Detection (Video)', annotated)
        key = cv2.waitKey(25) & 0xFF
        if key == ord('p'): paused = not paused
        elif key == ord('r'): current_pos = 0; paused = False
        elif key == 27: break
    cap.release()
    cv2.destroyAllWindows()

def run_image(path, detector):
    frame = cv2.imread(path)
    if frame is None: return
    blink_state = {'blink_count': 0, 'is_eye_closed': False}
    drowsy_state = {'consecutive_closed_frames': 0, 'consecutive_open_frames': 0,
'is_drowsy_alert_active': False}
    annotated, _, _ = process_frame_eyes_detection(frame, detector, blink_state,
drowsy_state)
    cv2.putText(annotated, "Static Image - Blinks N/A", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7,
(0, 0, 255), 2)
    cv2.imshow('Eyes Detection (Image)', annotated)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

# Module Execution
try: landmarker_detector = vision.FaceLandmarker.create_from_options(options)
except Exception as e: print(f"Failed to initialize: {e}"); return

while True:
    print("\n--- Eyes Detection Module ---")
    print("1. Live Webcam\n2. Video\n3. Image\nb. Back")
    choice = input("Choice: ").lower()

```

```

    if choice == '1': run_webcam(landmarker_detector)
    elif choice == '2': run_video(input("Video Path: "), landmarker_detector)
    elif choice == '3': run_image(input("Image Path: "), landmarker_detector)
    elif choice == 'b': break

# =====
# MODULE 3: FACE DETECTION (Pose Estimation & Hybrid Emotion Engine)
# =====
def face_detection_module():
    import cv2
    import numpy as np
    import mediapipe as mp
    from mediapipe.tasks.python import BaseOptions
    from mediapipe.tasks.python import vision
    from deepface import DeepFace
    import time
    import collections
    import os

    # Configuration for tuning the emotion engine
    DEEPFACE_INFERENCE_INTERVAL = 10 # Only run deepface every 10 frames (it's slow)
    EMOTION_HISTORY_DURATION = 5 # Keep track of mood for last 5 seconds
    MODEL_PATH = "face_landmarker.task"

    # Indices to calculate head pose
    NOSE_TIP_INDEX = 1
    LEFT_EYE_LANDMARKS_CENTRAL = [33, 133]
    RIGHT_EYE_LANDMARKS_CENTRAL = [362, 359]

    # Weights and thresholds for combining DeepFace analysis with MediaPipe blendshapes
    EMOTION_MIN_THRESHOLD = {"angry": 10, "fear": 10, "surprise": 12, "sad": 12, "happy": 20, "neutral": 50}
    EMOTION_BOOST = {"angry": 2.8, "fear": 2.8, "surprise": 3.0, "sad": 2.5, "happy": 1.2, "neutral": 0.15}
    EMOTION_HISTORY_WEIGHT = {"angry": 4.0, "fear": 4.0, "surprise": 3.5, "sad": 3.0, "happy": 2.0, "neutral": 0.2}
    BLENDSHAPE_CONFIRM_BONUS = 2.0

    # MediaPipe Blendshape Indices (scores from 0 to 1 indicating muscle movement)
    BS_BROW_INNER_UP = 1; BS_EYE_WIDE_L = 11; BS_EYE_WIDE_R = 12; BS_JAW_OPEN = 25
    BS_MOUTH_SMILE_L = 44; BS_MOUTH_SMILE_R = 45; BS_MOUTH_FROWN_L = 40;
    BS_MOUTH_FROWN_R = 41
    BS_MOUTH_UPPER_UP_L = 48; BS_MOUTH_UPPER_UP_R = 49; BS_NOSE_SNEER_L = 27;

```

BS_NOSE_SNEER_R = 28

```
def get_bs(blendshapes, index):
    """Helper to safely extract a blendshape score."""
    if blendshapes and index < len(blendshapes): return blendshapes[index].score
    return 0.0

def get_eye_midpoint(landmarks, eye_indices, img_w, img_h):
    """Averages the provided indices to find the center of an eye."""
    xs = [landmarks[i].x * img_w for i in eye_indices if i < len(landmarks)]
    ys = [landmarks[i].y * img_h for i in eye_indices if i < len(landmarks)]
    return (int(np.mean(xs)), int(np.mean(ys))) if xs else None

def estimate_head_pose(landmarks_list, img_w, img_h):
    """
    Estimates pose by comparing the nose's position relative to the center of both eyes.
    If the nose crosses a threshold distance from the center, the head is turned.
    """
    if not landmarks_list: return "Forward"
    nose = landmarks_list[NOSE_TIP_INDEX]
    nose_x, nose_y = int(nose.x * img_w), int(nose.y * img_h)

    left_mp = get_eye_midpoint(landmarks_list, LEFT_EYE_LANDMARKS_CENTRAL, img_w,
img_h)
    right_mp = get_eye_midpoint(landmarks_list, RIGHT_EYE_LANDMARKS_CENTRAL, img_w,
img_h)
    if not left_mp or not right_mp: return "Forward"

    avg_eye_x, avg_eye_y = (left_mp[0] + right_mp[0]) // 2, (left_mp[1] + right_mp[1]) // 2
    # Thresholds: 3% of width, 5% of height
    h_thresh, v_thresh = img_w * 0.03, img_h * 0.05

    res_h = "Right" if nose_x < avg_eye_x - h_thresh else "Left" if nose_x > avg_eye_x + h_thresh
else ""
    res_v = "Up" if nose_y < avg_eye_y - v_thresh else "Down" if nose_y > avg_eye_y + v_thresh
else ""

    combined = f"{res_h}-{res_v}".strip("-")
    return combined if combined else "Forward"

def get_blendshape_bonuses(blendshapes):
    """Reads specific facial muscles to confirm an emotion (e.g. Jaw open + eyes wide = Surprise)."""
    bonuses = {}
```

```

    jaw = get_bs(blendshapes, BS_JAW_OPEN)
    brow_in = get_bs(blendshapes, BS_BROW_INNER_UP)
    smile = (get_bs(blendshapes, BS_MOUTH_SMILE_L) + get_bs(blendshapes,
BS_MOUTH_SMILE_R)) / 2
    frown = (get_bs(blendshapes, BS_MOUTH_FROWN_L) + get_bs(blendshapes,
BS_MOUTH_FROWN_R)) / 2
    wide = (get_bs(blendshapes, BS_EYE_WIDE_L) + get_bs(blendshapes, BS_EYE_WIDE_R)) / 2
    upper_lip = (get_bs(blendshapes, BS_MOUTH_UPPER_UP_L) + get_bs(blendshapes,
BS_MOUTH_UPPER_UP_R)) / 2
    sneer = (get_bs(blendshapes, BS_NOSE_SNEER_L) + get_bs(blendshapes,
BS_NOSE_SNEER_R)) / 2

```

```

    if brow_in > 0.25 or frown > 0.2: bonuses["sad"] = BLENDSHAPE_CONFIRM_BONUS
    if smile > 0.4: bonuses["happy"] = BLENDSHAPE_CONFIRM_BONUS
    if jaw > 0.4 and wide > 0.3: bonuses["surprise"] = BLENDSHAPE_CONFIRM_BONUS
    if wide > 0.3 and jaw < 0.4 and brow_in > 0.2: bonuses["fear"] =
BLENDSHAPE_CONFIRM_BONUS
    if upper_lip > 0.2 and sneer > 0.1: bonuses["angry"] = BLENDSHAPE_CONFIRM_BONUS
    return bonuses

```

```

def pick_emotion(raw_scores, blendshapes, last_state):
    """Combines DeepFace CNN output with MediaPipe geometry to pick the winning emotion."""
    boosted = {e: s * EMOTION_BOOST.get(e, 1.0) for e, s in raw_scores.items() if s >=
EMOTION_MIN_THRESHOLD.get(e, 0)}
    if not boosted: boosted = raw_scores

```

```

    geo_bonuses = get_blendshape_bonuses(blendshapes)
    for e, b in geo_bonuses.items():
        if e in boosted: boosted[e] *= b

```

```

    winner = max(boosted, key=boosted.get) # Pick highest scoring emotion
    return winner.capitalize(), raw_scores.get(winner.lower(), 0.0)

```

```

def _process_frame(frame, detector, frame_count, emotion_history, last_state,
force_deepface=False):
    h, w, _ = frame.shape
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    mp_img = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
    results = detector.detect(mp_img)

```

```

    curr_emo = last_state["emotion"]
    curr_conf = last_state["confidence"]
    pose = "Forward"

```

```

if results.face_landmarks:
    landmarks = results.face_landmarks[0]
    pose = estimate_head_pose(landmarks, w, h)

    # DeepFace is computationally heavy, so only run it periodically or if forced (images)
    if force_deepface or (frame_count % DEEPFACE_INFERENCE_INTERVAL == 0):
        try:
            # Crop face tightly to give DeepFace an easier job
            xs = [lm.x for lm in landmarks]; ys = [lm.y for lm in landmarks]
            x1, y1 = int(max(min(xs)*w-20, 0)), int(max(min(ys)*h-20, 0))
            x2, y2 = int(min(max(xs)*w+20, w)), int(min(max(ys)*h+20, h))

            analysis = DeepFace.analyze(frame[y1:y2, x1:x2], actions=['emotion'],
enforce_detection=False, silent=True)
            bs = results.face_blendshapes[0] if results.face_blendshapes else None
            curr_emo, curr_conf = pick_emotion(analysis[0]['emotion'], bs, last_state)
            emotion_history.append((time.time(), curr_emo))
        except: pass

    # Draw green mesh points
    for pt in landmarks: cv2.circle(frame, (int(pt.x*w), int(pt.y*h)), 1, (0, 255, 0), -1)

    # Remove old emotions from history (sliding window approach)
    now = time.time()
    while emotion_history and now - emotion_history[0][0] > EMOTION_HISTORY_DURATION:
        emotion_history.popleft()

    mood = "N/A"
    if emotion_history:
        scores = collections.defaultdict(float)
        for _, emotion in emotion_history: scores[emotion] +=
EMOTION_HISTORY_WEIGHT.get(emotion.lower(), 1.0)
        mood = max(scores, key=scores.get)

    cv2.putText(frame, f"Emotion: {curr_emo} ({curr_conf:.1f}%)", (10, 30), 1, 1.5, (0, 255, 255), 2)
    cv2.putText(frame, f"Pose: {pose}", (10, 70), 1, 1.5, (0, 255, 0), 2)
    cv2.putText(frame, f"Recent Mood: {mood}", (10, 110), 1, 1.5, (255, 255, 0), 2)

    return frame, emotion_history, {"emotion": curr_emo, "confidence": curr_conf}

# Module execution logic...
if not os.path.exists(MODEL_PATH): print(f"Error: {MODEL_PATH} not found."); return

```

```

# Enable blendshapes output here!
options = vision.FaceLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=MODEL_PATH),
    output_face_blendshapes=True, num_faces=1
)
detector = vision.FaceLandmarker.create_from_options(options)
history = collections.deque()
state = {"emotion": "Neutral", "confidence": 0.0}
count = 0

while True:
    print("\n--- Face Module ---\n1. Webcam\n2. Video\n3. Image\n0. Exit")
    choice = input("Select: ")
    if choice == '0': break
    elif choice == '1':
        cap = cv2.VideoCapture(0)
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret: break
            frame, history, state = _process_frame(cv2.flip(frame, 1), detector, count, history, state)
            cv2.imshow("Face Det", frame)
            count += 1
            if cv2.waitKey(1) & 0xFF == 27: break
        cap.release(); cv2.destroyAllWindows()
    # Choices 2 and 3 omitted for brevity, logic is identical to Module 1 & 2
detector.close()

```

```

# =====
# MODULE 4: HAND DETECTION (Finger Counting & Gesture Game)
# =====
def hand_detection_module():
    import cv2, numpy as np, time, random, os, urllib.request, math
    import mediapipe as mp
    from mediapipe.tasks import python as mp_python
    from mediapipe.tasks.python import vision as mp_vision

    HandLandmarker = mp_vision.HandLandmarker
    HandLandmarkerOptions = mp_vision.HandLandmarkerOptions

    # Auto-downloads hand model if missing
    MODEL_URL =

```



```
"https://storage.googleapis.com/mediapipe-models/hand_landmarker/hand_landmarker/float16/1/hand_landmarker.task"
```

```
MODEL_PATH = "hand_landmarker.task"
```

```
if not os.path.exists(MODEL_PATH): urllib.request.urlretrieve(MODEL_URL, MODEL_PATH)
```

```
# Connections between joints to draw the skeleton
```

```
HAND_CONNECTIONS = frozenset([  
    (0, 1), (1, 2), (2, 3), (3, 4), (0, 5), (5, 6), (6, 7), (7, 8),  
    (5, 9), (9, 10), (10, 11), (11, 12), (9, 13), (13, 14), (14, 15),  
    (15, 16), (13, 17), (17, 18), (18, 19), (19, 20), (0, 17)  
])
```

```
# Indices for the very tip of the fingers, and the middle joint (PIP)
```

```
TIP_IDS = [4, 8, 12, 16, 20]
```

```
PIP_IDS = [3, 6, 10, 14, 18]
```

```
GESTURE_MAP = {0: ("Fist", "[FIST]"), 1: ("One", "[1]"), 2: ("Peace", "[V]"), 3: ("Three", "[3]"), 4: ("Four", "[4]")}
```

```
def count_fingers(lms: list, side: str) -> tuple:
```

```
    """
```

```
    Determines which fingers are open by measuring distance from the wrist (lm 0).
```

```
    If the fingertip is further away from the wrist than the middle joint, the finger is open.
```

```
    """
```

```
    ext = []
```

```
    # Thumb requires slightly different geometry reference (from pinky knuckle 17)
```

```
    d_thumb_tip = math.hypot(lms[4].x - lms[17].x, lms[4].y - lms[17].y)
```

```
    d_thumb_pip = math.hypot(lms[3].x - lms[17].x, lms[3].y - lms[17].y)
```

```
    ext.append(d_thumb_tip > d_thumb_pip)
```

```
    # Index, Middle, Ring, Pinky
```

```
    for i in range(1, 5):
```

```
        d_tip = math.hypot(lms[TIP_IDS[i]].x - lms[0].x, lms[TIP_IDS[i]].y - lms[0].y)
```

```
        d_pip = math.hypot(lms[PIP_IDS[i]].x - lms[0].x, lms[PIP_IDS[i]].y - lms[0].y)
```

```
        ext.append(d_tip > d_pip)
```

```
    return sum(ext), ext
```

```
def is_fingers_spread(lms) -> bool:
```

```
    """Checks if hand is wide open (High Five) or fingers together (Open Hand)."""
```

```
    tip_dist = math.hypot(lms[8].x - lms[20].x, lms[8].y - lms[20].y) # Index tip to pinky tip
```

```
    knuckle_dist = math.hypot(lms[5].x - lms[17].x, lms[5].y - lms[17].y) # Knuckle to knuckle
```

```
    return (tip_dist / knuckle_dist) > 1.6 if knuckle_dist != 0 else False
```

```

def classify_gesture(count: int, ext: list, lms: list) -> tuple:
    thumb, index, middle, ring, pinky = ext
    if thumb and not any([index, middle, ring, pinky]): return "Thumbs Up", "[TU]"
    if count == 5:
        return ("High Five", "[HI-5]") if is_fingers_spread(lms) else ("Open Hand", "[Hand-Closed]")
    return GESTURE_MAP.get(count, ("Unknown", "[?]"))

# Mini Game Logic implementation omitted for brevity, but it relies heavily on holding
# a gesture, comparing it to the `target` string, and tracking time via `time.time()`.

# ... Runners and UI ...

# =====
# MAIN ENTRY POINT
# =====
def main_menu():
    while True:
        print("\n--- SELECT MODULE ---")
        print("1. Lips\n2. Eyes\n3. Face\n4. Hands\nb. Exit")
        mode = input("Select: ").lower()
        if mode == 'b': break
        elif mode == '1': lips_detection_module()
        elif mode == '2': eyes_detection_module()
        elif mode == '3': face_detection_module()
        elif mode == '4': hand_detection_module()

if __name__ == "__main__":
    main_menu()

```
